Semester Project

Chanel Dobler

July 2026

Contents

1 SMC EX260-SDN1 DeviceNet **2**
- 1.1 Physical Switch Settings . . . 2
  - 1.1.1 Communication Speed . . . 2
  - 1.1.2 DeviceNet Address . . . 2
- 1.2 Messaging Protocol . . . 2
  - 1.2.1 Connection ID . . . 2
  - 1.2.2 Data Field . . . 3
  - 1.2.3 Setup Flowchart . . . 4
- 1.3 Duplicate MAC ID Check . . . 4
- 1.4 Master/Slave Connection Management . . . 5
- 1.5 Master's Explicit Request . . . 6
- 1.6 Interface . . . 8
- 1.7 Configuration . . . 8

2 Keller PAA-4LD I^2C **9**
- 2.1 Request Measurement . . . 9
- 2.2 Read Measurement . . . 9
- 2.3 Interface . . . 9
- 2.4 Configuration . . . 10

3 Autonics TM4-N2SB RS485 **10**
- 3.1 Function Codes . . . 10
- 3.2 Register Addresses . . . 10
- 3.3 Interface . . . 11
- 3.4 Configuration . . . 11

4 TCP/IP **11**

5 GUI **11**

6 Threads **12**

1 SMC EX260-SDN1 DeviceNet

1.1 Physical Switch Settings

1.1.1 Communication Speed

The communication speed is set with the physical switches No. 3 and 4. The default value is 125kbps.

Bitrate	Switch No.	
kbps	3	4
125	0	0
250	0	1
500	1	0

Table 1: Bitrates corresponding to Switch Settings

1.1.2 DeviceNet Address

The DeviceNet address is set with the physical switches No. 5 to 10. The default address setting is 63.

Address	Switch No.					
-	5	6	7	8	9	10
0	0	0	0	0	0	0
1	0	0	0	0	0	1
2	0	0	0	0	1	0
...				...		
62	1	1	1	1	1	0
63	1	1	1	1	1	1

Table 2: Addresses corresponding to Switch Settings

1.2 Messaging Protocol

A message consists of a connection ID and a data field. Wait $\geq 1s$ for a response when you send a message.

1.2.1 Connection ID

The connection ID counts 11 bits. The structure depends on the message group. Only group 2 is relevant for this project. The connection ID for group 2 contains the group ID (2 bits), the MAC ID (6 bits) and the message ID (3 bits).

	Group ID		MAC ID						Message ID		
Bit No.	10	9	8	7	6	5	4	3	2	1	0

Table 3: Group 2 Connection ID Structure

Message ID	Function
4	Master's Explicit Request
6	Master/Slave Connection Management
7	Duplicate MAC ID Check

Table 4: Relevant Group 2 Message IDs

1.2.2 Data Field

The data field may include some flags. The R flag indicates a request or response. The F flag indicates a complete or fragmented message. The X flag is a transaction ID echoed in a response to match a request.

Flag	0	1
R	Request	Response
F	Complete	Fragmented
X	-	-

Table 5: Data Field Flags

1.2.3 Setup Flowchart

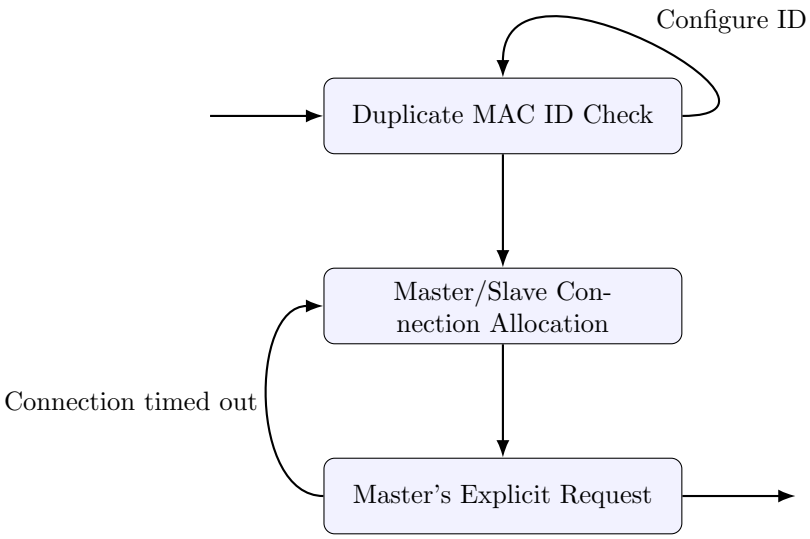


Figure 1: Communication Setup

1.3 Duplicate MAC ID Check

It's inevitable that two modules on the same link will be assigned the same address. It requires human intervention to configure a unique Pi address. The Pi address is written to the Connection ID MAC ID for the duplicate check.

Group ID		Pi Address						Message ID		
1	0	0	0	0	0	0	0	1	1	1

Table 6: Duplicate MAC ID Check Connection ID

Byte No.		
0	R	Physical Port Number
1		Vendor ID
2		
3		Serial Number
4		
5		
6		

Table 7: Duplicate MAC ID Check Data Field

Important: The duplicate MAC ID check has to be transmitted two consecutive times without receiving a subsequent request or response message before establishing a connection.

1.4 Master/Slave Connection Management

Group ID		DeviceNet Address						Message ID		
1	0	1	1	1	1	1	1	1	1	0

Table 8: Master/Slave Connection Management Connection ID

The Pi address is written to the MAC ID in the data field. The service code 0x4B indicates the allocation of a master/slave connection and 0x4C the release. The Class ID 3 directs the message at the DeviceNet object. There exists exactly one DeviceNet object per physical attachement to the DeviceNet. Therefore, the instance ID is always 1. The allocation choice byte denotes an explicit message and I/O connections from the predefined master/slave connection set. The acknowledge suppression bit has to be set to 0 for the release choice.

Byte No.			
0	F	X	MAC ID
1	R	Service Code [0x4B]	
2		Class ID [0x03]	
3		Instance ID [0x01]	
4		Allocation Choice	
5		Allocator's MAC ID	

Table 9: Master/Slave Connection Allocation Data Field

Byte No.			
0	F	X	MAC ID
1	R	Service Code [0x4C]	
2	Class ID [0x03]		
3	Instance ID [0x01]		
4	Release Choice		

Table 10: Master/Slave Connection Release Data Field

Bit No.	Connection
7	Reserved
6	Acknowledge Suppression
5	Cyclic
4	Change of State
3	Multicast Polling
2	Bit Strobed
1	Polled
0	Explicit Message

Table 11: Allocation/Release Choice

1.5 Master's Explicit Request

Explicit request messages are used to perform operations such as reading and writing attributes. The Pi address is written to the MAC ID in the data field. The service code 0x10 is used to write to a single attribute and the code 0x0E to read from a single attribute. The class ID 9 represents the discrete output points. The instance ID corresponds to the channel.

Group 2		DeviceNet Address						Message 4		
1	0	1	1	1	1	1	1	0	1	0

Table 12: Master's Explicit Request CID

Byte No.			
0	F	X	MAC ID
1	R	Service Code [0x10]	
2	Class ID [0x09]		
3	Instance ID		
4	Attribute ID [0x03]		
5	Value		

Table 13: Master's Explicit Set Single Value Data Field

Byte No.			
0	F	X	MAC ID
1	R	Service Code [0x0E]	
2	Class ID [0x09]		
3	Instance ID		
4	Attribute ID [0x03]		

Table 14: Master's Explicit Get Single Value Data Field

The class ID 4 represents the Assembly Object. The instance ID 37 corresponds to the 32 solenoid output points. The 4 value bytes act as a mask to set the values for all 32 channels at once.

Byte No.			
0	F	X	MAC ID
1	R	Service Code [0x10]	
2	Class ID [0x04]		
3	Instance ID [0x25]		
4	Attribute ID [0x03]		
5	Value		
6			
7			
8			

Table 15: Master's Explicit Set Multiple Values Data Field

Byte No.			
0	F	X	MAC ID
1	R	Service Code [0x0E]	
2	Class ID [0x04]		
3	Instance ID [0x25]		
4	Attribute ID [0x03]		

Table 16: Master's Explicit Get Multiple Values Data Field

1.6 Interface

Signature	Description
`duplicate_check()`	Execute duplicate MAC ID check
`connect()`	Allocate master/slave connection
`disconnect()`	Release the master/slave connection
`get_status(id)`	Get status bit for a valve
`get_status_all()`	Get a list of status bits for all valves
`open(id)`	Open a valve
`open_multiple(idx)`	Open a list of valves
`open_all()`	Open all valves
`close(id)`	Close a valve
`close_multiple(idx)`	Close a list of valves
`close_all()`	Close all valves
`start()`	Perform duplicate check and connect
`stop()`	Close all valves and disconnect

Table 17: Function Signatures and Descriptions

1.7 Configuration

SMC ID	Pi ID	Bitrate
63	0	125000

Table 18: DeviceNet Configuration

Name	ID
Reactor 1	1
Reactor 2	2
Reactor 3	3
Reactor 4	4
Reactor 5	5
Reactor 6	15
Reactor 7	14
Reactor 8	13
Reactor 9	12
Reactor 10	11
Line 1	6
Line 2	10
Vacuum	7
Trap	8
H2	9
???	16
Vortex	17
-	18
Lift	19

Table 19: Valves Configuration

2 Keller PAA-4LD I^{2}C

2.1 Request Measurement

The command to request a measurement is 0xAC. Wait $\geq 8ms$ before reading the measurement.

2.2 Read Measurement

The response contains the status and the measurement in 3 bytes. The status byte carries a checksum error indicator at bit 2. The two data bytes hold the most and least significant byte. The two data bytes are stitched together and then interpreted with $(p - 16384) * (p_{\max} - p_{\min})/32768 + p_{\min}$ for the pressure. The absolute pressure ranges from $p_{\min} = 0$ bar to $p_{\max} = 3$ bar.

2.3 Interface

Signature	Description
`get_pressure(id)`	Get pressure from a sensor
`get_pressure_all()`	Get pressures from all sensors

Table 20: Function Signatures and Descriptions

2.4 Configuration

Reactor No.	Address
Reactor 1	0x50
Reactor 2	0x51
Reactor 3	0x52
Reactor 4	0x53
Reactor 5	0x54
Reactor 6	0x55
Reactor 7	0x56
Reactor 8	0x57
Reactor 9	0x58
Reactor 10	0x59

Table 21: Keller Configuration

3 Autonics TM4-N2SB RS485

3.1 Function Codes

Code	Function
1	Read Coil Status
3	Read Holding Registers
4	Read Input Registers
5	Force Single Coil
6	Preset Single Register

Table 22: Relevant Function Codes

3.2 Register Addresses

Code	Address for Channel i	Function
1/5	0x0002 * (i-1)	Read/Write Status
3/6	0x03E8 * (i-1)	Read/Write Set Temperature
4	0x03E8 + 6 * (i-1)	Read Temperature

Table 23: Functions

Important: Run=0, Stop=1

3.3 Interface

Signature	Description
`get_temperature(id)`	Get temperature for a channel
`get_temperature_multiple(idx)`	Get temperatures for a list of channels
`get_temperature_all()`	Get temperatures for all channels
`set_temperature(id, val)`	Set target temperature for a channel
`set_temperature_multiple(idx, val)`	Set target temperature for a list of channels
`get_set_temperature(id)`	Read target temperature for a channel
`get_status(id)`	Return status bit for a channel
`run(id)`	Start operation for a single channel
`stop(id)`	Stop operation for a single channel
`run_multiple(idx)`	Start operation for a list of channels
`stop_multiple(idx)`	Stop operation for a list of channels

Table 24: Function Signatures and Descriptions

3.4 Configuration

Port	Address	Baudrate
/dev/ttySC0	1	9600

Table 25: Instrument Configuration

Name	ID
Trap	1
Oven 1	2
Oven 2	3

Table 26: Channel Configuration

4 TCP/IP

5 GUI

The GUI consists of two tabs. The first tab shows the control panel, the pressure output and a samples table. The second tab is used to display and edit configurations.

Include graphics

6 Threads

Every device has its own worker thread to pull data from the device regularly. Procedures have their own threads to work through the script in the background. All communication to the devices is handled through the device workers to avoid interleavings from multiple messages at the same time. The procedure workers add method requests to the device worker queue.

Name	Type	Description
`run`	Method	Execute requests
`request`	Method	Queue methods
`stop`	Method	Pause worker
`cache`	Field	Store latest data
`requests`	Field	Store method queue
`update`	Signal	Send latest data

Table 27: Worker Class Description

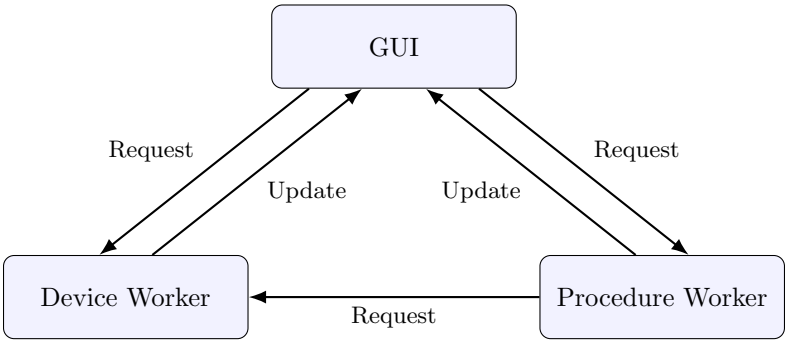


Figure 2: Worker and GUI Interactions